DAX Technical Consistency Audit and Orchestration Plan

Author: Manus AI
Date: May 2026
Project: DAX — Reasoning / Governance / Recursive Audit Engine
Scope: Architecture, Terminology, Workflow, and Deployment Consistency

---

1. Executive Summary

This report is a technical consistency audit and orchestration blueprint for the **DAX** project. The user intends to connect DAX with **Manus** (an autonomous AI agent platform) and **Kubernetes (K8s)** for deployment, but the framing "add K8s somehow to Manus for DAX" reveals two fundamental category errors that, if left uncorrected, will result in an undeployable system.

Category Error 1: Kubernetes is not a plugin for Manus. Kubernetes is a cluster-level infrastructure platform that manages containerized workloads. Manus is an application-level workflow agent. They operate at entirely different layers of the stack and do not have a parent-child relationship.

Category Error 2: Kubernetes cannot be introduced before Docker. K8s exclusively orchestrates containers. If DAX services are not yet containerized, K8s has nothing to manage.

The correct mental model is a layered stack: **Code → Docker (containerize) → Docker Compose (local integration) → Kubernetes (deploy) → Manus (coordinate the build/deploy workflow from above)**. Manus sits outside the cluster as an intelligent coordinator, not inside it as a runtime.

This report provides a corrected system model, a full inconsistency audit, a phased implementation plan, a Kubernetes object map, a Manus prompt pack, a minimal repository structure, and security notes. The single most important action the user should take right now is to build a working `/health` endpoint for the DAX API service — not to configure Kubernetes.

---

2. Corrected System Model

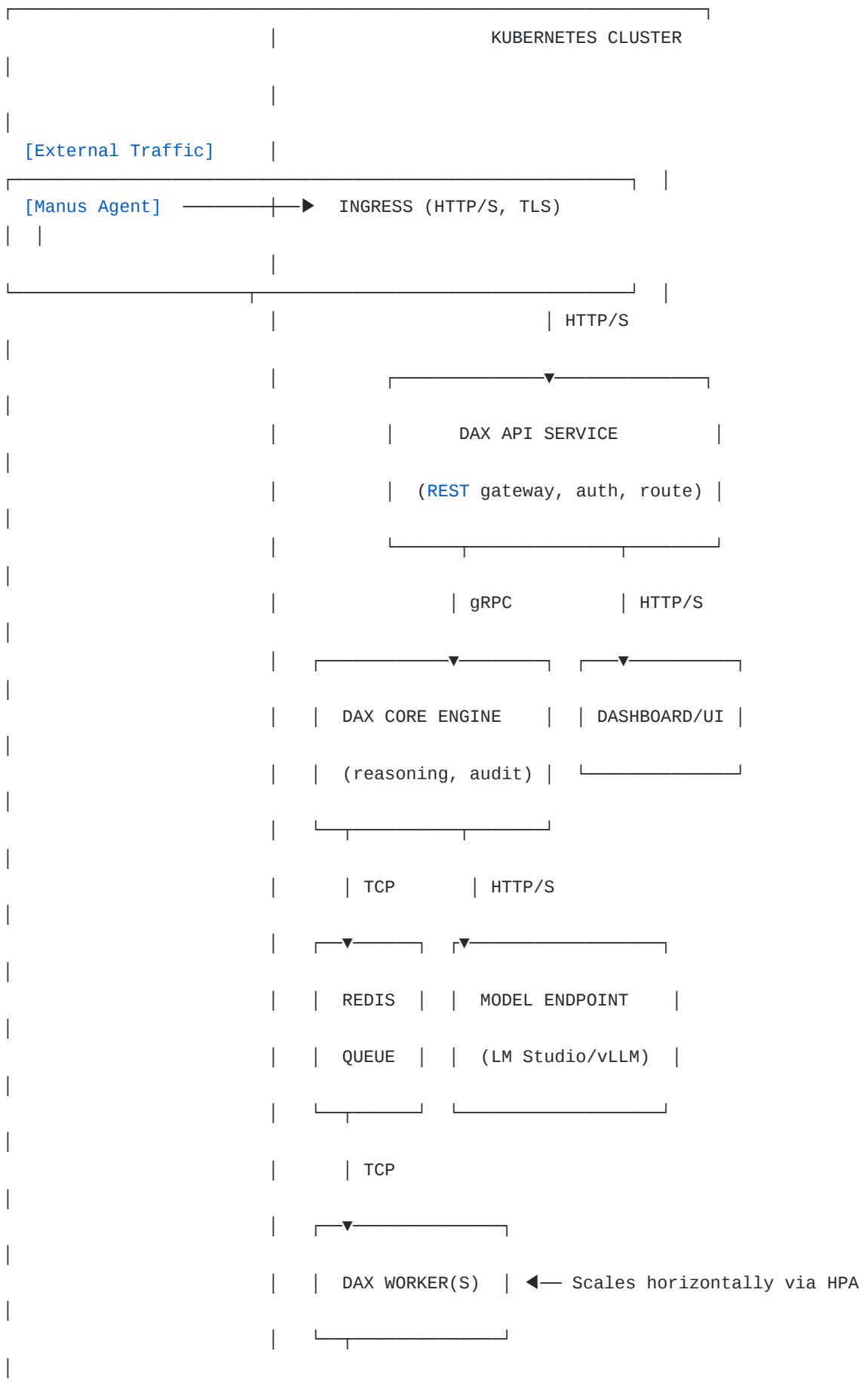
The table below defines the precise role of every component in the DAX architecture, what each component is not, and its interface boundary with other components.

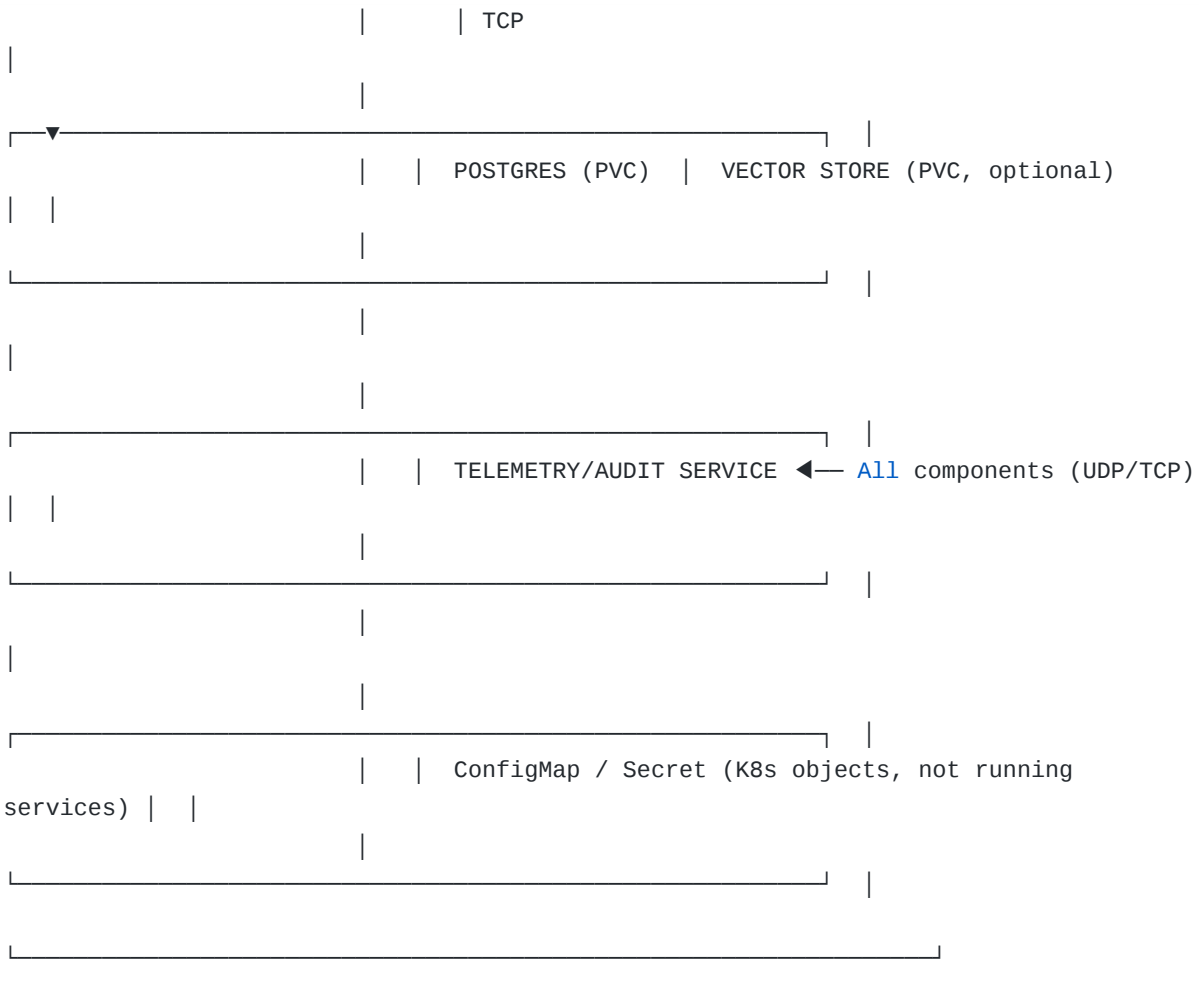
Component	What It IS	What It Is NOT	Interface Boundary
DAX (Core Engine)	Reasoning, governance, and recursive audit engine. Enforces access policy and reasons through vulnerabilities before data reaches an LLM.	A general-purpose LLM. A storage layer. A container orchestrator.	Exposes REST/gRPC API to DAX API service. Reads from Postgres/Vector Store. Dispatches tasks to Workers via Redis.
DAX API Service	The public-facing HTTP gateway into the DAX system. Handles auth, routing, and request validation.	The core reasoning logic itself.	Receives HTTP from Ingress and Manus. Calls DAX Core via gRPC.
DAX Worker Service	Executes discrete audit tasks dispatched by the Core Engine. Scales horizontally.	The orchestration layer.	Consumes tasks from Redis queue. Writes results to Postgres.
DAX Telemetry Service	Collects logs, metrics, and audit trails from all DAX components.	A primary data store for business logic.	Receives data via UDP/TCP from all services. Forwards to Prometheus/Grafana/ELK.
Dashboard/UI	Web-based interface for monitoring, configuration, and audit review.	A backend service or processing engine.	Calls DAX API over HTTP/S.
Manus	Autonomous AI agent and workflow coordinator. Generates manifests, triggers deploys, audits configs, and coordinates build steps via task prompts.	A runtime container orchestrator. A K8s plugin. A data store.	Calls DAX API over HTTP/S. Optionally calls K8s API via a scoped `ServiceAccount`.
Kubernetes	Deployment and container orchestration layer. Manages pod lifecycle, scaling, networking, and self-healing.	A development environment. A code runner. A Manus plugin.	Manages container images produced by Docker. Exposes services via Ingress.
Docker	Containerization layer. Packages DAX services and their dependencies into portable, reproducible images.	A container orchestration platform. A VM.	Produces images consumed by Kubernetes.

Component	What It IS	What It Is NOT	Interface Boundary
Redis	Asynchronous task queue and message broker between the Core Engine and Workers. High-throughput, in-memory.	A primary persistent database.	TCP connections from DAX Core and DAX Worker.
Postgres	Persistent relational storage for audit results, configuration, and user data. ACID-compliant.	An in-memory cache. A vector store.	TCP connections from DAX Core, API, and Worker via standard SQL drivers.
Vector Memory Store	Optional. Stores vector embeddings for semantic search and agent memory (e.g., pgvector, Chroma, Weaviate).	A traditional relational database.	HTTP or TCP from DAX Core Engine.
LM Studio / Model Endpoint	Local or containerized inference runtime for LLMs. Provides an OpenAI-compatible HTTP API.	A cloud service. A training platform.	HTTP/S from DAX Core Engine.

Reference Architecture Diagram

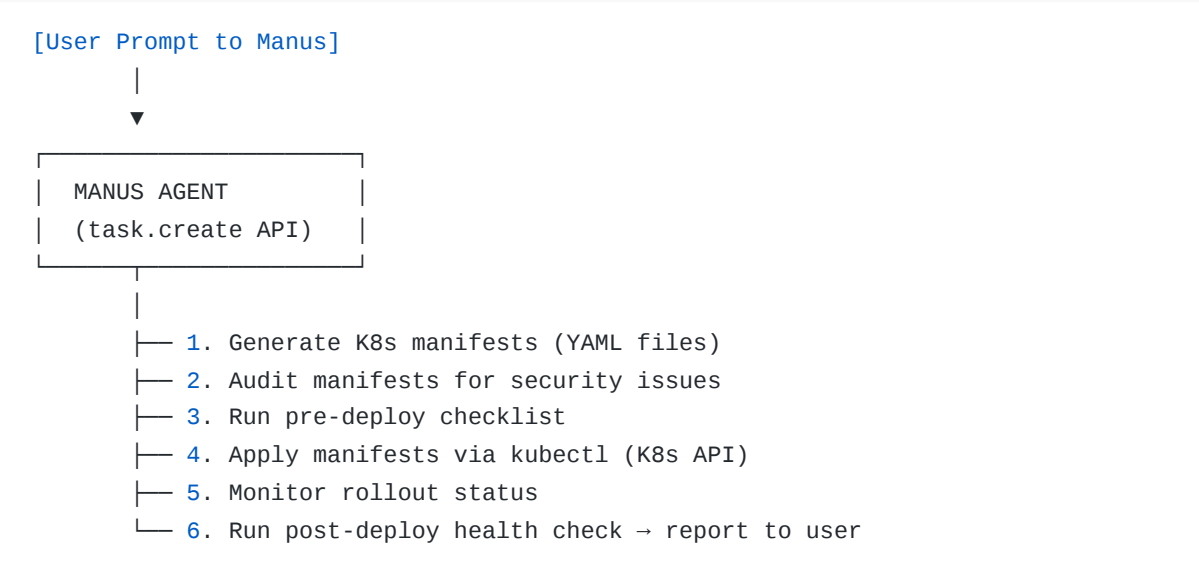
The following ASCII diagram shows the full DAX system within a Kubernetes cluster boundary, with Manus as an external coordinator.


KUBERNETES CLUSTER
[External Traffic]
[Manus Agent]
INGRESS (HTTP/S, TLS)
HTTP/S
DAX API SERVICE
(REST gateway, auth, route)
gRPC
HTTP/S
DAX CORE ENGINE
(reasoning, audit)
DASHBOARD/UI
TCP
HTTP/S
REDIS
QUEUE
MODEL ENDPOINT
(LM Studio/vLLM)
TCP
DAX WORKER(S)
Scales horizontally via HPA



Manus Orchestration Flow Diagram

Manus operates *outside* the cluster as a coordinator. It does not live inside K8s as a running service.



---

3. Inconsistency Audit Table

ID	Category	Description	Root Cause	Impact if Unresolved	Recommended Fix
INC-001	Architecture	Treating K8s as a Manus plugin.	Misunderstanding of K8s as infrastructure, not application-level.	Impossible deployment; wasted engineering effort.	Redefine: K8s hosts DAX services; Manus interacts via API from outside the cluster.
INC-002	Deployment	Attempting to use K8s before Docker/containerization.	Lack of awareness that K8s exclusively manages containers.	Deployment failure; K8s cannot run bare code.	Containerize all DAX components with Dockerfiles before writing any K8s manifests.
INC-003	Workflow	Overbuilding infrastructure before a health-check service exists.	Premature optimization; focus on orchestration before viability.	No way to verify DAX runs; cascading failures during deploy.	Build a minimal `/health` endpoint first. Prove the service runs before containerizing.
INC-004	Security	No secret management strategy.	Absence of a defined approach for API keys and DB passwords.	High risk of credential leakage and unauthorized access.	Use K8s Secrets or external manager (HashiCorp Vault). Never commit secrets to Git.
INC-005	Architecture	Mixing local LM Studio with cloud K8s assumptions.	Conflating local dev environment with distributed cloud infrastructure.	Cloud pods cannot reach a local LM Studio instance without complex tunneling.	Deploy model endpoint inside K8s (e.g., vLLM) or use a managed cloud LLM API.

ID	Category	Description	Root Cause	Impact if Unresolved	Recommended Fix
INC-006	Terminology	Conflating Manus orchestration with K8s orchestration.	Ambiguous use of the word “orchestration” for two different layers.	Misaligned team expectations; incorrect architecture decisions.	Define: “Workflow Orchestration” (Manus) vs. “Container Orchestration” (K8s).
INC-007	Architecture	Missing service boundary definitions.	Monolithic thinking applied to a distributed system.	Tightly coupled components that cannot scale independently.	Define REST/gRPC API contracts between all DAX components before building.
INC-008	Security	No namespace or RBAC plan.	Treating the K8s cluster as a flat, single environment.	A compromised pod can access all cluster resources.	Create a `dax-system` namespace and implement strict RBAC policies.
INC-009	Deployment	No observability plan before production.	Blind deployments without logging, metrics, or tracing.	Inability to diagnose failures or audit DAX reasoning processes.	Integrate Prometheus, Grafana, and centralized logging before any production deploy.
INC-010	Architecture	Ambiguous integration point between Manus and DAX.	Vague requirement to “orchestrate with Manus” without a defined protocol.	Manus and DAX remain disconnected; no trigger mechanism exists.	Design a specific webhook or API polling mechanism for Manus to submit tasks to DAX API.

---

4. Phased Implementation Plan

The phases below must be executed in strict sequence. Each phase has a defined entry criterion; do not advance until the exit criterion is met.

Phase 0: Define DAX Services and Boundaries

Goal: Establish a shared, documented understanding of every DAX component, its responsibilities, and its API contracts before writing a single line of code.

Entry Criteria: Project intent is understood at a high level.

Key Tasks:

1. List all DAX services: API, Core Engine, Worker, Telemetry, UI, Redis, Postgres, Vector Store, Model Endpoint.
2. Write a one-paragraph responsibility statement for each service.
3. Define the API contract (REST or gRPC) for each inter-service boundary.
4. Draw a data flow diagram showing how a single audit request travels through the system.
5. Identify all external dependencies and their network requirements.

Deliverables: Service inventory document, boundary diagram, draft API specs.

Exit Criteria: All services are named, their responsibilities are documented, and API contracts are agreed upon.

Common Pitfalls: Vague service definitions; skipping the data flow diagram; not accounting for external dependencies.

Phase 1: Local DAX API + Health Endpoint

Goal: Build the minimal viable DAX API service locally with a working health endpoint.

Entry Criteria: Phase 0 is complete.

Key Tasks:

1. Initialize the `dax-api` project directory with a chosen framework (e.g., FastAPI, Express).
2. Implement a `GET /health` route returning `{"status": "ok", "version": "0.1.0"}`.
3. Add basic structured logging (JSON format).
4. Write a unit test for the health endpoint.

5. Commit to version control.

Deliverables: Runnable local service, passing unit tests, committed codebase.

Exit Criteria: `curl http://localhost:8080/health` returns `200 OK`.

Common Pitfalls: Over-engineering Phase 1 with business logic; skipping tests; not using structured logging from the start.

Phase 2: Dockerize DAX Services

Goal: Package each DAX service into a reproducible Docker image.

Entry Criteria: Phase 1 is complete; at least one service runs locally.

Key Tasks:

1. Write a `Dockerfile` for `dax-api` using a pinned base image (e.g., `python:3.11-slim`, not `python:latest`).
2. Build the image locally: `docker build -t dax-api:0.1.0 .`
3. Run the container and verify the health endpoint: `docker run -p 8080:8080 dax-api:0.1.0`
4. Repeat for `dax-core`, `dax-worker`, and `dax-telemetry`.
5. Push images to a private container registry.

Deliverables: `Dockerfile` per service; images successfully built and pushed.

Exit Criteria: All service images build without errors and health checks pass inside containers.

Common Pitfalls: Using `:latest` tags; running as root inside containers; including secrets in the image.

Phase 3: Docker Compose Local Integration

Goal: Wire all DAX services together locally using Docker Compose to validate the full stack before touching Kubernetes.

Entry Criteria: All service images build successfully (Phase 2 complete).

Key Tasks:

1. Write `docker-compose.yml` defining all services, Redis, and Postgres.
2. Use environment variables (`.env` file, never committed) for all credentials.

3. Define a named network for inter-service communication.
4. Add health checks for Redis and Postgres.
5. Run `docker compose up` and verify the full stack is operational.

Deliverables: Working `docker-compose.yml`; full local stack running.

Exit Criteria: All services start, pass health checks, and can communicate with each other.

Common Pitfalls: Hardcoding credentials in `docker-compose.yml`; not using named volumes for Postgres data; skipping health checks.

Phase 4: Local Kubernetes Deployment

Goal: Translate the Docker Compose stack into Kubernetes manifests and deploy to a local cluster (Minikube or kind).

Entry Criteria: Phase 3 is complete; Docker Compose stack is stable.

Key Tasks:

1. Install Minikube or kind locally.
2. Write K8s manifests: `Namespace`, `Deployment`, `Service`, `ConfigMap`, `Secret`, `PVC` for each component.
3. Apply manifests: `kubectl apply -f infra/k8s/`
4. Verify pods are running: `kubectl get pods -n dax-system`
5. Test the health endpoint through the K8s Service.

Deliverables: K8s YAML manifests in `infra/k8s/`; local cluster running DAX.

Exit Criteria: All pods are in `Running` state; health endpoint is reachable via K8s Service.

Common Pitfalls: Committing Secrets to Git; using `:latest` image tags; forgetting PVCs for Postgres/Redis.

Phase 5: Manus-Assisted Build/Deploy Workflow

Goal: Use Manus to automate manifest generation, security audits, and deploy coordination.

Entry Criteria: Phase 4 is complete; local K8s deployment is stable.

Key Tasks:

1. Create a Manus project with a system instruction defining the DAX architecture context.
2. Use Manus to generate K8s manifests for a new service (see Prompt Pack below).
3. Use Manus to audit existing manifests for security issues.
4. Configure a webhook so Manus can notify DAX API on task completion.
5. Document the Manus prompt pack in `manus/prompts/`.

Deliverables: Manus prompt pack; webhook integration; automated manifest generation workflow.

Exit Criteria: Manus can generate a valid K8s manifest and audit an existing one without manual intervention.

Phase 6: Cloud/Production Kubernetes

Goal: Deploy DAX to a managed cloud Kubernetes cluster (EKS, GKE, or AKS) with full security and observability.

Entry Criteria: RBAC is configured; Secrets management is in place; Ingress TLS is configured; Prometheus/Grafana are deployed; all Phase 5 workflows are stable.

Key Tasks:

1. Provision a managed K8s cluster.
2. Configure Ingress with TLS via Cert-Manager.
3. Integrate external secret manager (Vault or cloud-native equivalent).
4. Deploy Prometheus and Grafana for observability.
5. Run a full security audit before going live.
6. Configure HPA for `dax-worker` and `dax-api`.

Deliverables: Production cluster running DAX; monitoring dashboards; security audit report.

Exit Criteria: All services are healthy; TLS is active; secrets are not in Git; alerts are configured.

---

5. Manus Orchestration Prompt Pack

Manus is the workflow coordinator. The following prompts are exact task instructions to give Manus. Each assumes the relevant prerequisites are in place.

Prompt 1: Generate K8s Manifests for a DAX Service

Prerequisites: Docker image exists in registry; service name and port are known.

> *Generate Kubernetes manifests for the `dax-worker` service in the `dax-system` namespace. The service uses image `your-registry/dax-worker:1.0.0`, listens on port 8081, and requires environment variables `REDIS_HOST` (from ConfigMap `dax-config`) and `POSTGRES_PASSWORD` (from Secret `dax-secrets`). Set CPU requests to `200m`, limits to `500m`. Memory requests `256Mi`, limits `512Mi`. Include a liveness probe on `GET /health` and a readiness probe on `GET /ready`. Output separate YAML files for Deployment, Service (ClusterIP), and HorizontalPodAutoscaler (min 2, max 10, target CPU 70%).*

Expected Output: `dax-worker-deployment.yaml`, `dax-worker-service.yaml`, `dax-worker-hpa.yaml`

Prompt 2: Audit a K8s Manifest for Security Issues

Prerequisites: Manifest file is available.

> *Audit the attached `dax-api-deployment.yaml` for Kubernetes security best practices. Check for: containers running as root, missing `securityContext`, use of `:latest` image tags, missing resource limits, sensitive data in environment variables (not from Secrets), privileged containers, missing readiness/liveness probes, and outdated API versions. Produce a Markdown report with findings categorized as Critical, High, Medium, or Low severity, and provide a corrected YAML snippet for each Critical finding.*

Expected Output: `security_audit_report.md` with severity-ranked findings and corrected YAML snippets.

Prompt 3: Pre-Deploy Checklist

Prerequisites: K8s cluster is accessible; manifests are ready.

> *Run a pre-deployment checklist for the `dax-worker` service before deploying to the `dax-system` namespace. Verify: (1) Docker image `your-registry/dax-worker:1.0.0` exists in the registry; (2) K8s manifest YAML is syntactically valid; (3) ConfigMap `dax-config` exists in `dax-system`; (4) Secret `dax-secrets` exists in `dax-system`; (5) the cluster has sufficient CPU and memory headroom; (6) Redis and Postgres services are reachable from within the namespace. Report PASS or FAIL for each item.*

Expected Output: `pre_deploy_checklist.md` with PASS/FAIL status per item.

Prompt 4: Coordinate a Rolling Deploy

Prerequisites: Pre-deploy checklist passed; updated manifest is ready.

> *Apply the updated `dax-api-deployment-v2.yaml` to the `dax-system` namespace using a rolling update strategy. Monitor the rollout with `kubectl rollout status`. If any pod fails its readiness probe within 3 minutes, automatically roll back using `kubectl rollout undo`. Report the final status, duration, and any rollback actions taken.*

Expected Output: `rolling_deploy_report.md` with deployment status and rollback events if any.

Prompt 5: Post-Deploy Health Check

Prerequisites: Deployment is complete.

> *Execute a post-deployment health check for all services in the `dax-system` namespace. For each service: (1) verify all pods are in `Running` state; (2) call the `/health` endpoint and expect `200 OK`; (3) check for any `OOMKilled` or `CrashLoopBackOff` events in the last 10 minutes; (4) confirm CPU and memory usage is within limits. Produce a summary report.*

Expected Output: `post_deploy_health_report.md` with per-service status.

Prompt 6: Generate Docker Compose from Service Definitions

Prerequisites: Service definitions document from Phase 0 is available.

> *Read the attached `dax-service-definitions.md` and generate a `docker-compose.yml` file for local development. Include services: `dax-api`, `dax-core`, `dax-worker`, `dax-telemetry`, `redis:7-alpine`, and `postgres:15-alpine`. Use a named bridge network `dax-net`. Mount a `.env` file for all credentials (do not hardcode). Add health checks for Redis and Postgres. Map `dax-api` to host port 8080.*

Expected Output: `docker-compose.yml`

Prompt 7: Scan for Hardcoded Secrets

Prerequisites: Repository is available.

> *Scan the entire `dax-orchestration/` repository for hardcoded secrets, API keys, passwords, and tokens. Check all `.py`, `.ts`, `.js`, `.yaml`, `.yml`, `.env`, and `.json` files. Flag any string matching patterns for: AWS keys, OpenAI API keys, Postgres passwords, Redis auth strings,*

> *JWT secrets, and private keys. Produce a report of all findings with file path, line number, and severity. Do not print the actual secret values.*

Expected Output: `secret_scan_report.md` with file paths and line numbers.

Prompt 8: Generate Helm Chart Skeleton for DAX

Prerequisites: K8s manifests from Phase 4 are complete.

> *Generate a Helm chart skeleton for the DAX system. The chart should be named `dax`, with a top-level `Chart.yaml`, `values.yaml`, and templates for: `deployment.yaml`, `service.yaml`, `configmap.yaml`, `secret.yaml`, `ingress.yaml`, and `hpa.yaml`. Parameterize image tags, replica counts, resource limits, and all environment variables through `values.yaml`. Include a `_helpers.tpl` with standard label helpers.*

Expected Output: A complete `helm/dax/` directory structure.

Prompt 9: Create DAX Service Boundary Definition Document

Prerequisites: Phase 0 service inventory is complete.

> *Create a formal service boundary definition document for the DAX system. For each service (`dax-api`, `dax-core`, `dax-worker`, `dax-telemetry`, `dax-ui`), document: (1) primary responsibility, (2) inbound interfaces (protocol, port, caller), (3) outbound interfaces (protocol, port, target), (4) data it owns, (5) data it reads but does not own, (6) failure mode and impact on other services. Format as a Markdown document with one section per service.*

Expected Output: `dax-service-boundaries.md`

Prompt 10: Produce a Dependency Graph of DAX Services

Prerequisites: Service boundary definition document is complete.

> *Read the attached `dax-service-boundaries.md` and produce a dependency graph of all DAX services. Show which services depend on which other services, and label each edge with the protocol and data type. Output the graph in two formats: (1) a Mermaid diagram (for rendering in GitHub/Notion), and (2) a plain ASCII diagram. Also produce a table listing each service's direct dependencies and the consequence of each dependency being unavailable.*

Expected Output: Mermaid diagram, ASCII diagram, and dependency impact table.

---

6. Kubernetes Object Map

The following table maps every K8s object type to its DAX application, with key configuration notes and warnings.

K8s Object	Applied To	Key Fields	Warnings
Namespace	All DAX resources	`name: dax-system`	All objects must specify this namespace.
Deployment	`dax-api`, `dax-core`, `dax-worker`, `dax-telemetry`, `dax-ui`	`replicas`, `image`, `resources`, `livenessProbe`, `readinessProbe`	Never use `:latest` image tag. Always set resource requests and limits.
Service (ClusterIP)	All internal services, Redis, Postgres, Vector Store	`type: ClusterIP`, `port`, `targetPort`	Default for all internal services. Do not expose Redis or Postgres externally.
Service (LoadBalancer)	`dax-api` (production only)	`type: LoadBalancer`	Incurs cloud provider cost. Use Ingress instead where possible.
Ingress	`dax-api`, `dax-ui`	`host`, `path`, `tls`	Always configure TLS. Use Cert-Manager for automated certificate management.
ConfigMap	All services	Non-sensitive config: `LOG_LEVEL`, `REDIS_HOST`, `POSTGRES_HOST`	Never store passwords, API keys, or tokens here.
Secret	All services	Sensitive data: `POSTGRES_PASSWORD`, `LM_STUDIO_API_KEY`, `JWT_SECRET`	**Never commit to Git.** Base64 encoded, not encrypted by default. Use Sealed Secrets or Vault in production.

K8s Object	Applied To	Key Fields	Warnings
PersistentVolumeClaim	Postgres, Redis, Vector Store	`accessModes: ReadWriteOnce`, `storage: 10Gi`	Without a PVC, all database data is lost when the pod restarts.
HorizontalPodAutoscaler	`dax-api`, `dax-worker`	`minReplicas: 2`, `maxReplicas: 10`, `targetCPUUtilizationPercentage: 70`	Requires Metrics Server to be installed in the cluster.
Job / CronJob	Audit batch runs, telemetry flush	`schedule: "0 * * * *"`, `backoffLimit: 3`	Use Jobs for one-time tasks; CronJobs for scheduled tasks.
ServiceAccount + RBAC	Manus agent (if cluster access needed), internal pods	`rules: [verbs: [get, list, create], resources: [deployments]]`	Grant only minimum necessary permissions. Never use `cluster-admin` for Manus.
NetworkPolicy	All namespaces	`podSelector`, `ingress/egress rules`	Default-deny all, then explicitly allow required traffic paths.

Key YAML Snippets

Namespace:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dax-system
  labels:
    app.kubernetes.io/name: dax
```

DAX API Deployment:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dax-api
  namespace: dax-system
spec:
  replicas: 2
  selector:
    matchLabels:
      app: dax-api
  template:
    metadata:
      labels:
        app: dax-api
    spec:
      serviceAccountName: dax-api-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
      containers:
        - name: dax-api
          image: your-registry/dax-api:1.0.0   # Never use :latest
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: dax-config
            - secretRef:
                name: dax-secrets
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 15
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
```

Secret (template — never commit actual values):

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: dax-secrets
  namespace: dax-system
type: Opaque
data:
  POSTGRES_PASSWORD: <base64-encoded-value>   # Use: echo -n 'yourpassword' | base64
  LM_STUDIO_API_KEY: <base64-encoded-value>
  JWT_SECRET: <base64-encoded-value>
# WARNING: Never commit this file with real values to Git.
# Use Sealed Secrets, External Secrets Operator, or Vault instead.
```

---

7. Minimal DAX Repository Structure

```
dax-orchestration/
│
├── services/                       # All DAX application code
│   ├── dax-api/                    # HTTP gateway service
│   │   ├── src/                    # Application source code
│   │   ├── Dockerfile              # Container build instructions
│   │   └── requirements.txt        # Python deps (or package.json for Node)
│   ├── dax-core/                   # Reasoning and governance engine
│   │   ├── src/
│   │   └── Dockerfile
│   ├── dax-worker/                 # Async audit task workers
│   │   ├── src/
│   │   └── Dockerfile
│   ├── dax-telemetry/              # Observability and audit log service
│   │   ├── src/
│   │   └── Dockerfile
│   └── dax-ui/                     # Web dashboard (React/Vue/etc.)
│       ├── src/
│       ├── package.json
│       └── Dockerfile
│
├── infra/                          # Infrastructure as Code
│   ├── docker-compose.yml          # Phase 3: local integration
│   └── k8s/                        # Phase 4/6: Kubernetes manifests
│       ├── namespace.yaml
│       ├── configmap.yaml
│       ├── secret.yaml.template    # Template only — no real values
│       ├── dax-api/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   └── hpa.yaml
│       ├── dax-worker/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   └── hpa.yaml
│       ├── postgres/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   └── pvc.yaml
│       ├── redis/
│       │   ├── deployment.yaml
│       │   ├── service.yaml
│       │   └── pvc.yaml
│       └── ingress.yaml
│
├── manus/                          # Manus orchestration assets
│   └── prompts/                    # Prompt packs (see Section 5)
```

```
│       ├── generate-manifests.md
│       ├── audit-manifest.md
│       ├── pre-deploy-checklist.md
│       └── post-deploy-health.md
│
├── config/                          # Configuration templates (no secrets)
│   ├── .env.template                # Template for local .env file
│   └── dax-config-values.yaml      # Non-sensitive K8s ConfigMap values
│
├── scripts/                         # Automation scripts
│   ├── build.sh                     # Build all Docker images
│   ├── deploy.sh                    # Apply K8s manifests
│   └── health-check.sh              # Verify all services are healthy
│
├── docs/                            # Architecture and decision records
│   ├── architecture.md
│   └── service-boundaries.md
│
├── .gitignore                       # Exclude .env, *.secret, kubeconfig
├── README.md                        # Project overview and quick start
├── Makefile                         # Shortcuts: make build, make deploy, make
health
└── docker-compose.yml               # (symlink or copy of infra/docker-compose.yml)
```

---

8. Risks and Security Notes

Risk	Severity	Mitigation
Credentials committed to Git	Critical	Use `.gitignore` to exclude `.env` and `*.secret` files. Use pre-commit hooks (e.g., `git-secrets`, `detect-secrets`). Use K8s Secrets or Vault for all credentials.
Cluster over-privilege for Manus	Critical	Create a dedicated `ServiceAccount` for Manus with a `Role` granting only `get`, `list`, `create`, and `update` on `deployments` and `pods`. Never grant `cluster-admin`.
Redis and Postgres exposed externally	Critical	Use `ClusterIP` service type only. Implement `NetworkPolicy` to restrict access to only the services that need it.
DAX audit results tampered	Critical	Store audit logs in append-only storage. Implement cryptographic signing of audit records. Separate the role of audit trigger from audit result access.
No TLS on Ingress	High	Use Cert-Manager with Let's Encrypt or a private CA. Never serve DAX API over plain HTTP in any environment beyond local development.
LM Studio running locally while K8s is cloud	High	Cloud pods cannot reach a local LM Studio. Either containerize the model endpoint inside K8s, use a managed LLM API, or establish a secure tunnel — but document the data boundary clearly.
`:latest` image tags in production	High	Pin all images to immutable version tags (e.g., `dax-api:1.2.3`). Enforce this in CI/CD pipelines.
No observability before production	High	Deploy Prometheus and Grafana before any production traffic. Configure alerts for pod restarts, high error rates, and CPU/memory saturation.
Manus agent acting autonomously on cluster	High	Define explicit boundaries: Manus may read cluster state and generate manifests, but any `apply` or `delete` action must require human confirmation via `task.confirmAction`.
Supply chain vulnerabilities in base images	Medium	Scan all images with Trivy or Grype in CI. Use minimal base images (e.g., `distroless`, `alpine`). Maintain a Software Bill of Materials (SBOM).

9. Deferred Questions

These questions do not block the report but must be answered before advancing to Phase 4 or beyond.

Infrastructure:

1. Will the K8s cluster be self-managed or a managed service (EKS, GKE, AKS)?
2. What storage class will be used for PersistentVolumeClaims?
3. Will the model endpoint (LM Studio) be containerized inside K8s or remain local?
4. Is a private container registry already provisioned?

Service Design:

1. What is the exact API contract between `dax-api` and `dax-core`? REST or gRPC?
2. What triggers a DAX audit — a human user, a scheduled CronJob, or an external event?
3. Is the vector memory store required for Phase 1, or is it deferred?
4. What is the expected throughput for `dax-worker` (tasks per second)?

Security:

1. Who is authorized to trigger a DAX audit? Is there a role hierarchy?
2. Where will K8s Secrets be managed — native K8s Secrets, Sealed Secrets, or an external Vault?
3. What is the data classification of information processed by the LLM endpoint?

Operational:

1. What is the target SLA for the `dax-api` service?
2. Who owns the on-call rotation for production incidents?
3. Is there a CI/CD pipeline already in place, or does one need to be built?
4. What is the retention policy for audit logs?

---

10. Prioritized Inconsistency Fix List

CRITICAL — Must fix before any deployment

ID	Description	Effort	Why It Matters
FIX-001	Redefine K8s role: infrastructure layer, not Manus plugin.	S	Without this, the architecture is conceptually broken and no deployment can proceed.
FIX-002	Containerize all DAX services with Dockerfiles before writing K8s manifests.	M	K8s cannot manage uncontainerized code. This is a hard prerequisite.
FIX-003	Build a working `/health` endpoint before any orchestration work.	S	Without a verifiable health signal, there is no way to confirm the service is running.
FIX-004	Define service boundaries and API contracts for all DAX components.	M	Without defined contracts, services cannot communicate reliably.

IMPORTANT — Must fix before production

ID	Description	Effort	Why It Matters
FIX-005	Implement K8s Secrets management; never commit credentials to Git.	S	Credential leakage is the most common and most damaging security failure.
FIX-006	Design `dax-system` namespace and RBAC policies.	M	Without namespace isolation, a compromised pod can access all cluster resources.
FIX-007	Resolve the LM Studio local-vs-cloud boundary.	M	Cloud pods cannot reach a local model endpoint without explicit, secure networking.
FIX-008	Deploy Prometheus and Grafana before production traffic.	M	Blind production deployments are a reliability anti-pattern.
FIX-009	Configure Ingress with TLS for all external-facing services.	S	Plain HTTP in production is unacceptable for a governance/audit engine.
FIX-010	Define Manus trust boundary: what it can do autonomously vs. what requires confirmation.	S	An over-privileged autonomous agent is a significant security risk.

LATER — Nice to have / optimization

ID	Description	Effort	Why It Matters
FIX-011	Generate a Helm chart for DAX to simplify multi-environment deployments.	L	Helm simplifies configuration management but is not required for initial deployment.
FIX-012	Implement HPA for `dax-worker` and `dax-api`.	S	Autoscaling is important for production efficiency but not for initial validation.
FIX-013	Add distributed tracing (Jaeger/Zipkin) for cross-service request visibility.	M	Valuable for debugging complex audit flows, but not a Day 1 requirement.
FIX-014	Generate and maintain a Software Bill of Materials (SBOM).	M	Important for supply chain security, but can be deferred until the stack is stable.
FIX-015	Implement NetworkPolicy default-deny with explicit allow rules.	M	Best practice for production security hardening; can be added after initial deployment.

---

11. Next Concrete Action

Stop configuring Kubernetes. Stop researching Manus integrations.

The single most important next action is to prove that the DAX API service runs at all.

3-Step Micro-Plan:

1. **Create the `dax-api` directory** in your repository. Initialize a minimal web server project (e.g., `fastapi` in Python or `express` in Node.js).
2. **Implement `GET /health`** returning `{"status": "ok", "service": "dax-api", "version": "0.1.0"}`. Run it locally and confirm `curl http://localhost:8080/health` returns `200 OK`.
3. **Write a `Dockerfile`** for the service. Build it with `docker build -t dax-api:0.1.0 .` and run it with `docker run -p 8080:8080 dax-api:0.1.0`. Confirm the health endpoint works inside the container.

Once this works, you have a deployable unit. Every subsequent phase — Docker Compose, Kubernetes, Manus orchestration — builds directly on this foundation.

References:

[1] Kubernetes Documentation. “Overview.” https://kubernetes.io/docs/concepts/overview/
[2] Manus API Documentation. “Task Lifecycle.” https://docs.manus.im/v2/task-lifecycle
[3] Docker Documentation. “Overview.” https://docs.docker.com/get-started/overview/
[4] HashiCorp Vault. “Secrets Management.” https://www.vaultproject.io/
[5] Cert-Manager. “Kubernetes Certificate Management.” https://cert-manager.io/docs/